Tarea 4

API REST, JSON y almacenamiento móvil
Desarrollo completo de las partes A, B y C

Parte A • 1. API REST, HTTP y JSON

Una API es una interfaz que permite que programas intercambien datos y operaciones. Una API REST organiza la interacción alrededor de recursos identificados por URL, como /productos/15. Aplica una interfaz uniforme y comunicación sin estado: cada solicitud contiene la información necesaria para procesarla. Puede utilizar caché y una arquitectura por capas. REST es un estilo arquitectónico; no es un protocolo ni exige usar JSON.

Petición HTTP: el método indica la operación; la URL identifica el recurso y puede incluir parámetros; los encabezados describen el mensaje o llevan credenciales; el cuerpo contiene los datos enviados cuando corresponde. Una consulta GET normalmente no lleva cuerpo.

Respuesta HTTP: incluye un código de estado, encabezados y, cuando corresponde, un cuerpo. El código informa el resultado. Una respuesta 204 no contiene cuerpo. Método y URL pertenecen a la solicitud; el código de estado pertenece a la respuesta.

```
POST /pedidos HTTP/1.1
Host: api.ejemplo.com
Authorization: Bearer TOKEN_DE_EJEMPLO
Content-Type: application/json
Accept: application/json

{"productoId":15,"cantidad":2}

HTTP/1.1 201 Created
Content-Type: application/json
Location: /pedidos/301

{"id":301,"productoId":15,"cantidad":2}
```

En este ejemplo la URL completa sería https://api.ejemplo.com/pedidos. Content-Type describe el formato del cuerpo; Accept indica el formato de respuesta deseado. Authorization transporta la credencial y Location señala el recurso creado.

JSON (JavaScript Object Notation) es un formato textual de intercambio independiente del lenguaje. Admite objetos, arreglos, cadenas, números, booleanos y null. Las claves y cadenas usan comillas dobles; no admite comentarios ni comas finales. Ejemplo: {"id":15,"disponible":true}. No incluye tipos especiales de fecha o de moneda: su representación debe acordarse en el contrato de la API. [1, 2]

Métodos y resultados

Parte A • 2. Semántica de HTTP

Método	Uso y comportamiento
GET	Obtiene una representación del recurso. Es seguro e idempotente: no solicita modificar datos y repetirlo no añade efectos.
POST	Envía datos para que el recurso los procese; suele crear un recurso subordinado. No es idempotente por definición: repetirlo podría crear duplicados.
PUT	Reemplaza por completo la representación del recurso en la URL indicada. Es idempotente. Puede crear el recurso si el contrato lo permite.
PATCH	Aplica cambios parciales. Su idempotencia depende del cambio: fijar un teléfono es idempotente; incrementar un contador puede no serlo.
DELETE	Solicita eliminar el recurso. Es idempotente en su efecto; una repetición puede responder 404 aunque la primera responda 204.

Código	Significado
200 OK	Operación correcta; normalmente devuelve una representación.
201 Created	Se creó un recurso; puede indicar su URL mediante Location.
204 No Content	Operación correcta sin cuerpo de respuesta.
400 Bad Request	La solicitud es inválida: por ejemplo, JSON mal formado.
401 Unauthorized	Faltan credenciales válidas o el token expiró. Se requiere autenticación válida.
403 Forbidden	El servidor rechaza la operación por falta de autorización. Iniciar sesión de nuevo no necesariamente lo resuelve.
404 Not Found	No se encontró el recurso solicitado.
500 Internal Server Error	Fallo inesperado del servidor al procesar la solicitud.

Los códigos y los cuerpos concretos forman parte del contrato de cada API. Las respuestas de éxito indicadas aquí siguen las convenciones de HTTP. [1]

Aplicación de HTTP

Parte A • 3. Método, éxito y justificación

Acción	Solicitud de ejemplo	Éxito y justificación
a) Listar productos	GET /productos	200 OK: devuelve el arreglo. Si no hay productos, puede responder 200 con [].
b) Crear un pedido	POST /pedidos	201 Created: crea un pedido nuevo; devuelve su ID y puede incluir Location.
c) Reemplazar todo el perfil	PUT /usuarios/7/perfil	200 OK si devuelve el perfil completo; 204 si no devuelve cuerpo. Se envían todos los campos reemplazables.
d) Cambiar solo el teléfono	PATCH /usuarios/7/perfil	200 OK si devuelve el perfil actualizado; 204 sin cuerpo. Solo se modifica el campo solicitado.
e) Eliminar dirección	DELETE /usuarios/7/direcciones/2	204 No Content: eliminación completada sin cuerpo.

Códigos de error esperados

f) Sesión expirada o token inválido: 401 Unauthorized. La credencial ya no acredita una sesión válida. La app puede intentar una renovación segura y, si falla, solicitar autenticación. No debe confundirlo con 403, que expresa una prohibición de acceso.

g) Producto inexistente: 404 Not Found. Por ejemplo, GET /productos/999999 cuando ese identificador no existe. No corresponde 500: la ausencia del recurso no es un fallo inesperado del servidor.

Ejemplo de actualización parcial

```
PATCH /usuarios/7/perfil HTTP/1.1
Host: api.ejemplo.com
Content-Type: application/json

{"telefono":"809-555-0100"}

HTTP/1.1 204 No Content
```

Este cuerpo supone que la API define JSON simple como formato de actualización parcial. Otras APIs usan JSON Patch o JSON Merge Patch y requieren sus tipos de contenido específicos.

Análisis del JSON

Parte B • 4. Tipos, estructura y modelos

```
{
  "id": 15,
  "nombre": "Acetaminofén 500 mg",
  "precio": 85.50,
  "disponible": true,
  "categorias": ["analgésico", "antipirético"],
  "sucursal": {"codigo": "STI-01", "ciudad": "Santiago"}
}
```

Campo	Tipo JSON	Representación Kotlin
id	number (valor entero)	Int
nombre	string	String
precio	number (valor decimal)	Double
disponible	boolean	Boolean
categorias	array de string	List<String>
sucursal	object	Sucursal
sucursal.codigo	string	String
sucursal.ciudad	string	String

a) JSON tiene un único tipo numérico; entero y decimal describen estos valores, no dos tipos JSON distintos. **b)** El documento raíz es un objeto; sucursal es un objeto anidado y categorias es un arreglo de dos cadenas. **c)** Los modelos solicitados se representan así en Kotlin:

```
data class Producto(
    val id: Int,
    val nombre: String,
    val precio: Double,
    val disponible: Boolean,
    val categorias: List<String>,
    val sucursal: Sucursal
)

data class Sucursal(
    val codigo: String,
    val ciudad: String
)
```

Double sirve para representar el ejemplo. En operaciones monetarias reales conviene usar centavos enteros o BigDecimal con deserialización definida, para evitar errores de redondeo binario. Todos los campos se consideran obligatorios porque el JSON dado los incluye.

Almacenamiento de la app

Parte C • 5a. Comparación de alternativas

Opción	Ventajas	Limitaciones	Aplicación
SharedPreferences / DataStore	Datos pequeños por clave. DataStore ofrece acceso asíncrono y actualizaciones consistentes.	Sin consultas relacionales. No cifra secretos automáticamente.	Tema, idioma, sucursal elegida y fecha de sincronización.
SQLite / Room	Consultas, índices, relaciones y transacciones. Funciona sin Internet; Room facilita el acceso a SQLite.	Requiere esquema, migraciones y sincronización. No equivale a cifrado automático.	Catálogo, líneas del carrito e historial en caché.
Archivos privados	Adecuados para imágenes, documentos y exportaciones.	Buscar y actualizar registros es más difícil. Hay que gestionar tamaño y limpieza.	Fotos de productos, recibos descargados y exportaciones.
Base en la nube, p. ej. Firebase	Datos compartidos entre dispositivos y respaldo remoto; algunos SDK tienen caché local.	Depende de red para confirmar cambios remotos; requiere reglas de acceso y resolver conflictos.	Catálogo oficial, pedidos confirmados e historial autorizado.

5b. Qué guardaría y por qué

Catálogo: fuente oficial en la nube o servidor, copia estructurada en Room y fotografías en archivos de caché. La API entrega precios y disponibilidad; la copia local permite navegar sin red.

Carrito: Room para conservar productos, cantidades y cambios pendientes de manera transaccional. Puede sincronizarse con el servidor al iniciar sesión. El precio y el stock se validan de nuevo antes de confirmar un pedido.

Preferencias: DataStore para idioma, tema y sucursal seleccionada. Son datos pequeños sin relaciones complejas.

Token de sesión: almacenamiento seguro respaldado por Android Keystore o Keychain de iOS. No se guarda en preferencias sin cifrar ni junto a las imágenes de caché.

Historial: fuente oficial en el servidor y copia local en Room, separada por usuario. Solo se muestran pedidos realmente confirmados. Se elimina o protege la copia al cerrar sesión para evitar exponerla a otra cuenta. [3, 4]

Sin conexión y seguridad

Parte C • 5c y 5d

5c. Estrategia de funcionamiento sin conexión

1. Inicio: mostrar inmediatamente el catálogo y el historial existentes en la base local, con su fecha de actualización. Si no hay caché, mostrar “Aún no hay datos guardados; conéctate para descargarlos”.

2. Actualización: consultar la API al abrir si la copia tiene más de 15 minutos, al actualizar manualmente y cuando vuelve la conexión. Este intervalo es una decisión propuesta para el caso. Guardar cada respuesta válida mediante una transacción y actualizar la interfaz desde la base local.

3. Caché: conservar productos, categorías, sucursales y últimos pedidos; guardar imágenes con límite de tamaño y eliminar las menos recientes. Si la descarga falla, mantener la última copia válida y avisar que puede estar desactualizada.

4. Sin Internet: permitir buscar en el catálogo guardado, consultar historial descargado y editar el carrito local. Mostrar “Sin conexión” y advertir que precio y disponibilidad no están confirmados. No presentar una compra como realizada hasta recibir confirmación del servidor.

5. Reconexión: sincronizar cambios pendientes con identificadores únicos para evitar duplicados. Revalidar precios, existencias y sesión; comunicar cambios al usuario antes de confirmar. Los datos oficiales de inventario y pedidos prevalecen sobre la copia local.

5d. Protección del token de sesión

Un token de acceso funciona como una credencial: quien lo obtiene puede actuar con sus permisos hasta que expire o sea revocado. Guardarlo en texto plano facilita su robo desde copias de respaldo, archivos expuestos o registros. La carpeta privada de la app por sí sola no sustituye el cifrado.

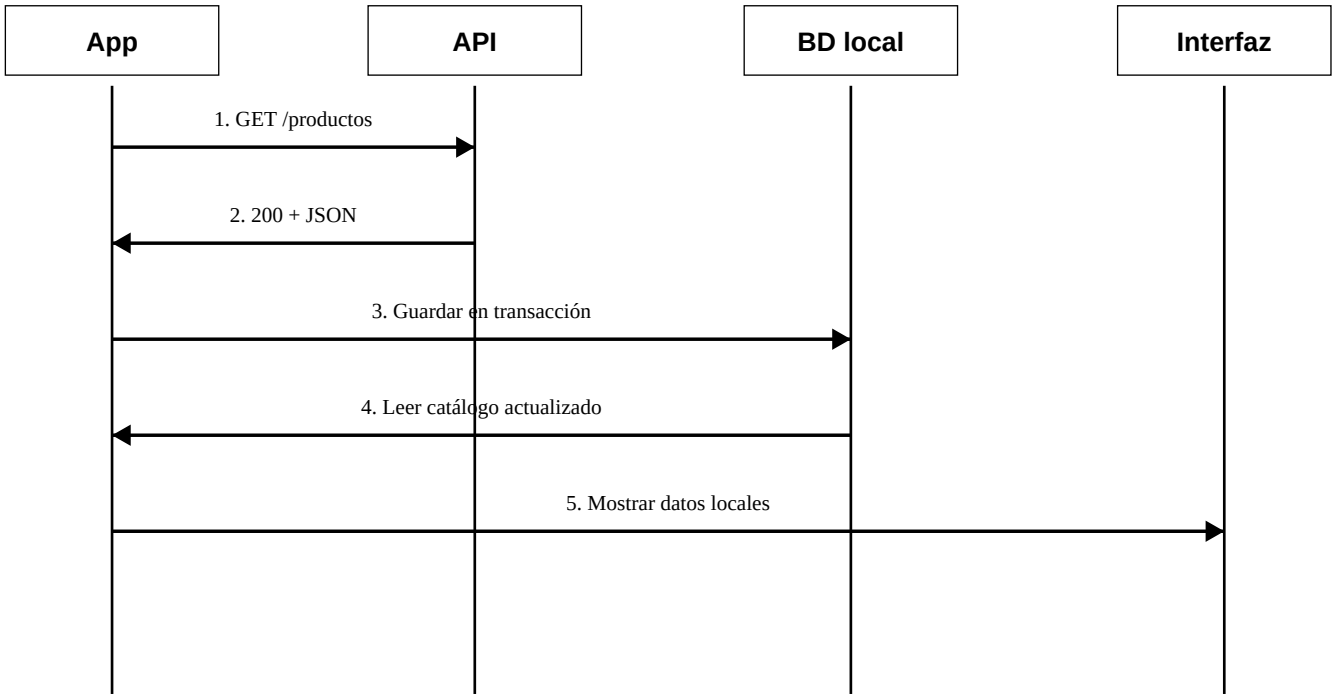
Android: generar una clave no exportable en Android Keystore y utilizarla para cifrar el token, por ejemplo con AES-GCM. Persistir únicamente el texto cifrado y el nonce; nunca la clave junto al token. Keystore protege las claves criptográficas, no almacena directamente cualquier cadena como si fuera una preferencia. [5]

iOS: guardar la credencial en Keychain con una política de accesibilidad adecuada, preferiblemente limitada al dispositivo y al estado desbloqueado cuando el uso lo permita. [6]

Usar HTTPS; excluir secretos de logs y repositorios; mantener el token de acceso de corta duración en memoria cuando sea posible y proteger el token de renovación. Al cerrar sesión se eliminan las credenciales locales y se solicita revocación al servidor. Ante 401 se renueva de forma controlada o se pide iniciar sesión. Ningún mecanismo elimina todo riesgo en un dispositivo comprometido.

Consulta del catálogo

Parte C • 5e. Diagrama de secuencia



La app coordina la API y el almacenamiento del dispositivo.

La secuencia solicitada app → API → base de datos local → interfaz se implementa con la app como coordinadora. La API no escribe directamente en SQLite del teléfono: responde a la app, y esta guarda la respuesta antes de actualizar la interfaz.

Flujo alternativo sin conexión: App → BD local: leer caché; BD local → App: datos guardados; App → Interfaz: catálogo almacenado y aviso “Sin conexión”. Si la API falla, se conserva el mismo flujo alternativo.

Referencias

1. RFC 9110: semántica de HTTP

2. RFC 8259: formato JSON

3. Android: almacenamiento con Room

4. Android: DataStore

5. Android Keystore

6. Apple: Keychain Services